

САНКТ-ПЕТЕРБУРГСКИЙ НАЦИОНАЛЬНЫЙ
ИССЛЕДОВАТЕЛЬСКИЙ УНИВЕРСИТЕТ ИТМО

Дисциплина: Бэк-энд разработка

ОТЧЁТ

Домашняя работа 4

Технический дизайн микросервисной архитектуры

Выполнил:
Прель Александр
Группа БР1.1

Проверил:
Добряков Д. И.

Санкт-Петербург
2026 г.

Задача

1. Разделить монолит на микросервисы с database-per-service.
2. Показать архитектуру и способы взаимодействия.
3. Разделить БД на обособленные базы.
4. Описать внутренние endpoints в OpenAPI.
5. Определить запросы, ответы и ошибки.
6. Сформулировать конкретные требования и шаги реализации.

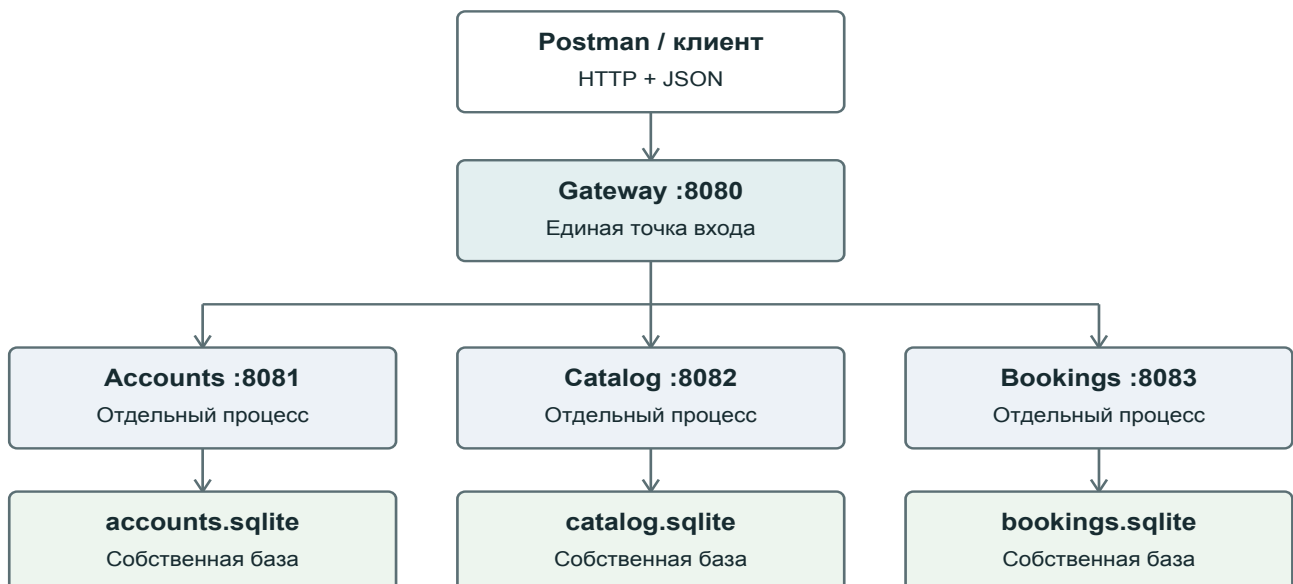
Ход работы

1. Границы ответственности

Компонент	Ответственность	Данные
Accounts	Регистрация, вход, профиль, JWT	users
Catalog	Рестораны, меню, кухни, столики, фото, отзывы и рейтинг	Восемь таблиц каталога
Bookings	Создание, история, отмена, занятость столиков	reservations
Gateway	Публичная точка входа, маршрутизация, готовность	Без БД

Регистрация и профиль используют одну запись User и принадлежат Accounts. Отзывы остаются в Catalog вместе с рейтингом ресторана: отдельный сервис отзывов для этого варианта не требуется. Каждый сервис имеет отдельный процесс и свою модель данных.

2. Архитектура



Gateway направляет /auth и /users в Accounts, /restaurants и /cuisines в Catalog, /reservations в Bookings. Catalog обращается к Accounts при создании отзыва. Bookings обращается к Accounts за профилем и к Catalog за столиками. Протокол: синхронный HTTP/JSON.

Все адреса находятся на 127.0.0.1. Порты по умолчанию: 8080-8083; фактические значения задаются в .env. Общий исходный пакет не означает общую БД: код сервисов не импортирует чужие entities и data-source.

3. Разделение БД

База	Таблицы и преобразования
accounts.sqlite	users. Удаляются обратные ORM-связи с Review и Reservation.
catalog.sqlite	restaurants, cuisines, restaurant_cuisines, tables, menu_categories, menu_items, restaurant_photos, reviews. Локальные FK сохраняются.
reviews	user_id становится логическим ID без FK к Accounts; user_name хранит снимок имени автора.
bookings.sqlite	reservations. user_id, restaurant_id, table_id являются логическими ID. Добавлены снимки restaurant_name и table_number.

Межбазовых JOIN и FK нет. Название ресторана и имя автора сохраняются на момент создания записи; последующее переименование не изменяет историю. Проверка занятости принадлежит только Bookings. Удаление пользователей и ресторанов текущим API не поддерживается; его добавление требует отдельной политики архивирования.

4. Контракты межсервисного взаимодействия

Метод	Запрос	Ответ
GET Accounts /internal/auth/verify	Без тела. X-Service-Key и Authorization: Bearer JWT	200: userId, firstName, lastName, email, phone, createdAt, updatedAt
GET Catalog /internal/restaurants/{restaurantId}/booking-context	Без тела. X-Service-Key. restaurantId: положительное целое	200: restaurantId, restaurantName, openingTime, closingTime, tables[]

Каждый элемент tables содержит tableId, tableNumber и capacity. Возвращаются только активные столики, отсортированные по вместимости и ID. Пустой массив допустим. Занятость времени этот метод не определяет.

Полная OpenAPI 3.0.3 спецификация: homeworks/hw4/internal-api.yaml. Она содержит обязательные поля, ограничения типов, схемы авторизации, примеры успеха и ошибок. Валидация Swagger CLI 4.0.4 прошла: internal-api.yaml is valid.

5. Авторизация и отказы

Accounts проверяет HS256 JWT с issuer restaurant-accounts, audience restaurant-api, сроком 3600 секунд и существование пользователя. Catalog и Bookings получают актуальный профиль по HTTP. Внутренние методы защищены SERVICE_KEY и не доступны через Gateway. Клиентские X-Service-Key и X-User-Id не проксируются.

HTTP	Смысл
400 / 422	Некорректный путь, синтаксис JSON или поля запроса

HTTP	Смысл
401 / 403	Недействительные credentials либо доступ к чужой брони
404 / 409	Объект отсутствует либо конфликт email, занятости или отмены
413 / 500	Тело более 64 КБ либо непредвиденная ошибка
503	Сервис недоступен, некорректный ответ зависимости или занятая БД
504	Истёк таймаут межсервисного запроса

Формат ошибки: {"error":{"code":"SERVICE_UNAVAILABLE","message":"Зависимый сервис недоступен."}}. Внутренний таймаут: 2 секунды. Gateway: 6,5 секунды. Повторы записи автоматически не выполняются. /ready проверяет три сервиса и возвращает 200 либо 503.

6. Согласованность бронирований

Bookings сначала проверяет пользователя и поля, затем получает столики из Catalog, после чего пишет только в свою БД. Проверку пересечения выполняет BEFORE INSERT/UPDATE триггер SQLite, поэтому два параллельных запроса не могут занять один столик на пересекающееся время. Соседние интервалы разрешены. Отмена выполняется условным UPDATE собственных активных записей.

Распределённая транзакция не нужна: изменяется одна база. При недоступном Catalog история остаётся доступной по сохранённым снимкам. При недоступном Accounts публичный каталог работает, защищённые операции возвращают 503. Потеря ответа после записи требует проверки истории перед повтором; idempotency-key пока отсутствует.

7. Шаги реализации

1. Зафиксировать внешний контракт и тесты Д33.
2. Создать labs/lab2 с существующими зависимостями.
3. Выделить Accounts и Catalog, убрать внешние ORM-связи.
4. Выделить Bookings с логическими ID и снимками.
5. Реализовать внутренние методы по OpenAPI и HTTP-клиенты.
6. Добавить Gateway, таймауты, /health и /ready.
7. Подготовить три новые БД и независимый seed каталога.
8. Проверить сценарии, конкурентную запись, отказы, перезапуски и схемы БД.

8. План переноса имеющихся данных

Текущий запуск использует новые базы; автоматического переноса нет. Для отдельного переноса: остановить запись, сделать согласованный снимок исходной SQLite, читать его без изменений; перенести пользователей и каталог с сохранением ID; заполнить снимки в отзывах и бронированиях; сверить количество строк, FK и пересечения; проверить новые сервисы; переключить клиент. После новых записей в микросервисах простой откат адреса без сверки данных недопустим.

Старые JWT ЛР1 потребуют повторного входа из-за новых issuer/audience и секрета. Управление столиками, события, контейнеризация и production-изоляция системных пользователей рассматриваются в следующих этапах.

Вывод

Спроектированы три микросервиса и шлюз, определены владельцы данных, внутренние HTTP-контракты, ошибки и шаги перехода. Дизайн реализован в LP2 и проверен на macOS. Полная документация с диаграммами: [homeworks/hw4/README.md](#); подтверждённые результаты: [labs/lab2/TEST_RESULTS.md](#).